

Group 6: Navya, Madeline, Faraja

Language to be used: Ruby

Programming Exponential Decay:

For this assignment, we initially asked Chat GPT to generate the code, but it was giving us issues. Instead, we asked Claude to generate the code to make a forgetting curve in python, which we downloaded as a .py file and ran through clicking the “Run Function” button on the Mac computer. We had to download VS Code to display the plot.

We then used the python code and told Claude to use it to generate a code for the forgetting curve in Ruby. It gave a code that was showing us the plot on terminal. We consulted a Computer Science friend to help us execute the code better and we then asked Claude to give us a separate plot in a .svg file.

Additionally, we went through the code Claude generated and removed any unnecessary code.

Navya used her previous experience as a CS major to help us break down the code and we asked Claude to break down the code and explain the process so we understood what each chunk did. Several changes or additions had to be made to the prompt, such as the version of Ruby the code was running in.

Navya’s Perspective: We used my Claude to generate the code. Since I was a CS major, it assumed I knew what to do, so we had to ask it to chunk it down and take us through the process. The problem with LLMs is that there is so much information you have to give, for example, which version the Ruby is running in, because there are certain commands that Claude gave me which can’t run on the version of Ruby I have. We used ChatGPT first and it wasn’t helping so we had to move to Claude, but I still feel like I have an output in front of me but I didn’t really understand how it came to be, what the code did. Writing a code on your own is a very different experience than just asking an LLM to generate it for you. I think it’s an art in itself to learn to give LLMs the right prompt.

Some of the Prompts used (Claude):

Use the `rubypplot` library to plot the curve instead of doing it in the terminal
<http://github.com/SciRuby/rubypplot>

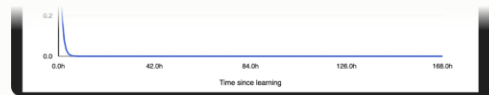
Read a page, ran a command >

I looked at the repo, and I want to flag something before we go further: **rubypplot is not in a good state to use for this.**

A few things from its README:

- It's explicitly labeled **alpha-stage** software (pre-1.0, not even a finished 0.1 release).
- It doesn't plot on its own — it requires you to separately download and install the **GR Framework** (a native C graphics library, not a gem), then manually set `GRDIR` and `GKS_FONTPATH` environment variables pointing to wherever you installed it.
- There's no simple "just `gem install rubypplot` and it works" path documented.

Given you're brand new to both Ruby and VS Code, and we already hit a wall with the `gruff / rmagick / ImageMagick` chain, I'd expect `rubypplot` to be a rougher version of that same problem — except this one's actively unmaintained-feeling and thinly documented, so if something breaks mid-install, there's less to go on to fix it.



Lets stick with what we have for now, but the plot is incorrect as shown,

Diagnosing a scale mismatch in the decay plot.

You're right that it doesn't look like a proper forgetting curve — but it's actually not a bug, it's the same scaling issue from before creeping back in. The default `S = 1.0` means the curve mathematically halves-and-more within the first hour, so across a 168-hour window it's basically a vertical drop to zero at the very left edge, then a flat line. The math is correct; the *default* number is just unrealistic for a week-long view.

Easiest fix: change the script's default so it looks right the moment you run it, without needing to remember flags.

Edited a file, ran a command >

That's the shape you want — the ASCII plot confirms it now spreads the decay across the full week instead of collapsing at the left edge. The default is now `S = 24` (a one-day memory-strength constant), so running the script with no flags at all will give you this curve.

Re-run it in VS Code the same way as before — `ruby ebbinghaus_forgetting_curve.rb` — and check `ebbinghaus_curve.svg` again; it should now look like the classic steep-drop-then-flatten forgetting curve instead of a cliff-edge.

give me the files

Presented 2 files

Both are below — the updated Ruby script and the SVG chart it generates (with the fixed default so it plots correctly out of the box).



Ebbinghaus forgetting curve
RB

Download



Ebbinghaus curve
Image · SVG

Download

Download all

📄 🔊 📄 🔄 2 days ago